

**КОНСТРУЮ
ПРОГРАМ
ЗАБЕЗПЕЧ**

ПРО ЩО КУРС

- **Python:** об'єктна модель, функції, ітератори/генератори
- **Java / C#:** ООП у керованих середовищах (CLR), поліморфізм, дженерики,
- **Дизайн:** SOLID, GRASP, шаблони

**ПЛАНУЄТЬСЯ ОРІЄНТОВНО 15 ЛАР
(АЛЕ ЦЕ НЕ ТО
30 БАЛІВ ВІНОСИТЬСЯ НА ІСПИТ**

ХТО ЧИТАЄ ЛЕКЦІЮ?

**ДЯЧУК РОСТИСЛАВ
ЛЮБОМИРОВИЧ**

-  PhD, асистент кафедри ПЗКС
-  Senior Java Developer,
monobank

ФІЛОСОФІЯ ТИПІЗАЦІЇ ВНУТРІШНІЙ ІНТЕРПРЕТАЦІЇ

**ZEN OF PYTHON • ТИПІЗАЦІЯ
МОДЕЛЬ ОБ'ЄКТІВ •**

ПЛАН ЛЕКЦІЇ

1. Zen of Python і внутрішній устрій
2. Типізація: динамічна, сувора, D
3. Модель об'єктів, LEGB, механіз

ДЕ ВЗЯТИ ПУТНО ПРАЦЮВА

- Сам Python: python.org/download
- IDE: [PyCharm](#)
- Текстовий редактор: [VS Code](#)
- Для самих дорослих і серйозн

ДЛЯ ТИХ ХТО ЛЮБИТ

тут мало бути посилання на встановлення `emacs`,
але прийдеться розбиратись самим

ZEN OF PYTHON (PEP

```
>>> import this
```

```
Beautiful is better than ugly.
```

```
Explicit is better than implicit.
```

```
Simple is better than complex.
```

```
Readability counts.
```

```
Errors should never pass silently.
```

КЛЮЧОВІ ПРИНЦИПИ

- **Beautiful / Readability** — індент
- **Explicit > Implicit** — обов'язков
- **One obvious way** — довго не бу
- **Errors never pass silently** — оск

EXPLICIT IS BETTER THAN IMPLICIT

```
>>> "2" + 2
TypeError: can only concatenate str (not "int") to str

>>> str(2) + "2"
'22'
```

ДО PYTHON 3.10

```
if status == "ok":  
    handle_ok()  
elif status == "error":  
    handle_error()  
else:  
    handle_default()
```

PYTHON 3.10+ — MAT

```
match status:
    case "ok": handle_ok()
    case "error": handle_error()
    case _: handle_default()
```

Лише коли pattern-matching справд

БАЙТКОД І ВІРТУАЛІЗАЦІЯ (PVM)

1. .py → байткод — компіляція в байткод
2. байткод → PVM — виконання інтерпретатором

DIS — ЗАГЛЯНУТИ В Б

```
import dis

def add(a, b):
    return a + b

dis.dis(add)

# LOAD_FAST a / LOAD_FAST b
# BINARY_ADD / RETURN_VALUE
```

Байткод кешується у `.рус` в `__рус`

ДИНАМІЧНІСТЬ ВИКОНАННЯ

```
def make_adder(n):  
    code = f"lambda x: x + {n}"  
    return eval(code)  
  
add5 = make_adder(5)  
add5(10)    # 15 — функція згенерована в рантаймі
```

АТРИБУТ МОЖНА Д КОЛИ

```
class Foo:
    pass

f = Foo()
f.new_attr = 42    # додано вже після створення
print(f.new_attr) # 42
```

GIL — GLOBAL INTERP

- Один потік виконує байткод од
- CPU-bound задачі → multipro

ДИНАМІЧНА ТИПІЗАЦІЯ

```
x = 5
type(x)      # <class 'int'>
x = "hello"
type(x)      # <class 'str'>
x = [1, 2, 3]
type(x)      # <class 'list'>
```

Тип належить значенню, а не змінній

ДЛЯ ПОРІВНЯННЯ: СТ

```
int x = 5;  
x = "hello";  
// error: cannot convert 'const char*' to 'int'
```

СУВОРА ТИПІЗАЦІЯ —

```
>>> "2" + 2
TypeError: can only concatenate str (not "int") to str
>>> int("2") + 2
4    # приведення явне
```

СЛАБКА ТИПІЗАЦІЯ –

```
"2" + 2    // "22" – неявне приведення  
"5" - 1    // 4 – а тут неявно приведено у інший бік!
```

JS непослідовна (то конкатенація,
свідомо цього у

DUCK TYPING

```
class Duck:
    def quack(self):
        print("Кря-кря!")

class Person:
    def quack(self):
        print("Імітую качку")

def make_it_quack(entity):
    entity.quack() # isinstance() не потрібен
```

ВИКЛИК

```
make_it_quack(Duck())    # Кря-кря!  
make_it_quack(Person()) # Імітую качку
```

ДЛЯ ПОРІВНЯННЯ: JAVA

```
interface Quacker {  
    void quack();  
}  
  
class Duck implements Quacker {  
    public void quack() {...}  
}
```

У Java об'єкт має явно реалізувати

LBYL — LOOK BEFORE

```
if hasattr(entity, "quack") and callable(entity.quack):  
    entity.quack()
```

EAFP — EASIER TO AS

```
try:  
    entity.quack()  
except AttributeError:  
    pass
```

Ідіоматичний Python. Пов'язано з
silently".

DUCK TYPING I PROTO

```
class Countdown:

    def __init__(self, n):
        self.n = n

    def __iter__(self):
        return self

    def __next__(self):
        if self.n <= 0:
            raise StopIteration

        self.n -= 1

        return self.n + 1

for x in Countdown(3):
    print(x)    # 3 2 1
```

ДИЗАЙН НА ОСНОВІ І

- `__iter__ / __next__` → ітерація
- `__len__` → `len(obj)`
- `__enter__ / __exit__` → `with`
- `__eq__ / __lt__` → порівняння

УСЕ В PYTHON — ОБ'Є

```
x = 42  
  
id(x)      # ідентичність (адреса)  
type(x)    # <class 'int'>  
  
x          # значення: 42
```

IDENTITY / TYPE / VALUE

- **identity** — незмінна впродовж життя
- **type** — визначає доступні операції
- **value** — залежить від мутабельності

МУТАБЕЛЬНІ VS НЕМ

- Мутабельні: `list`, `dict`, `set`
- Немутабельні: `int`, `str`, `tuple`,

МУТАБЕЛЬНІСТЬ: СП

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)  # [1, 2, 3, 4] — один і той самий об'єкт
```

ПЕРЕДАЧА АРГУМЕНТІВ

```
def modify(lst):  
    lst.append(99)    # видно зовні  
  
def reassign(n):  
    n = n + 1         # НЕ видно зовні  
  
modify([1, 2])        # список зміниться  
reassign(5)           # число — ні
```

Python передає "посилання на об'єкт"

АТРИБУТИ ТА `__dict__`

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(1, 2)
print(p.__dict__)    # {'x': 1, 'y': 2}
p.z = 99             # новий ключ у __dict__
```

ВИНЯТОК — `__slots__`

```
class Point:
    __slots__ = ("x", "y")

p = Point()
p.z = 1
# AttributeError — немає __dict__
```

ПРАВИЛО LEGB

Local → **Enclosing** → **Global** → **Built-in**

```
x = "global"

def outer():
    x = "enclosing"

    def inner():
        x = "local"

        print(x)      # -> "local"

    inner()

    print(x)          # -> "enclosing"

print(x)              # -> "global"
```

NONLOCAL

```
def counter():  
    count = 0  
  
    def increment():  
        nonlocal count  
        count += 1  
        return count  
    return increment  
  
c = counter()  
print(c()) # 1  
print(c()) # 2
```

GLOBAL

```
total = 0

def add_to_total(n):
    global total
    total += n
```

Без `nonlocal/global` — нова ЛС
ЗОВНІШНЬОС

UNBOUNDLOCALERROR

```
x = 10  
def broken():  
    print(x)    # UnboundLocalError!  
    x = 20
```

Присвоєння будь-де в тілі функції
для всієї функції

ВИПРАВЛЕННЯ

```
x = 10
def fixed():
    global x
    print(x)    # 10
    x = 20
```

ПРОСТІР ІМЕН КЛАСУ

```
class Example:
    attr = 10
    def method(self):
        # print(attr) -> NameError!
        print(self.attr)    # правильно
```

Доступ — лише явно: `self.attr`
`Example.attr`

ЩО ВІДБУВАЄТЬСЯ П

1. Перевірка `sys.modules` — вже
2. Пошук файлу — `sys.path`
3. Реєстрація в `sys.modules` — Д
4. Виконання коду модуля — згор

SYS.MODULS

```
import sys
print(sys.modules.keys())
# усі завантажені модулі процесу
```

Крок 3 — ключ до циклічних залеж

ЦИКЛІЧНІ ЗАЛЕЖНОСТІ

```
# module_a.py  
  
import module_b  
  
def func_a():  
    return module_b.func_b()
```

```
# module_b.py  
  
import module_a  
  
def func_b():  
    return module_a.func_a()
```

ЩО ВІДБУВАЄТЬСЯ module_

1. виконує `module_a.py`, реєструє
2. `import module_b` → починає м
3. `module_b: import module_a` —
не виконує повторно
4. `func_a` ще не визначена? → `At`
`ImportError`

РІШЕННЯ 1 — ВІДКЛА

```
def func_a():  
    import module_b  
    return module_b.func_b()
```

РІШЕННЯ 2 — РЕСТРУ

Спільна логіка виноситься в common
обидва мод

РІШЕННЯ 3 — ІМПОРТ ІМЕН

```
import module_b  
module_b.func_b()
```

РІШЕННЯ 4 — TYPE_C

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    import module_b

def f(b: "module_b.X"): ...
```

Тема	Ключова ідея
Zen of Python	Явність і чита синтаксис
Байткод + PVM	Компіляція в б
Динамічна типізація	Тип належить
Сувора типізація	Немає неявни
Duck Typing	Поведінка вах
Модель об'єкта	identity / type
LEGB	Local → Enclo

Тема	Ключова ідея
sys.modules	Кеш до виконання залежності

МАГІЧНІ М
ДЕСКРИП
ДЕКОРАТ
КОНТЕК
МЕНЕДЖ

ОСНОВИ ООП У PYTHON І

ПЛАН ЛЕКЦІЇ

1. Базові поняття ООП у Python
2. Магічні методи (Dunder Method)
3. Дескриптори та властивості (property)
4. Декоратори функцій і класів
5. Контекстні менеджери (with)

КЛАС ТА ЕКЗЕМПЛЯР

```
class Dog:
    pass

rex = Dog()
buddy = Dog()
```

rex і buddy — два різні об'єкти, хоч

__init__ TA self

```
class Dog:

    def __init__(self, name, breed):

        self.name = name

        self.breed = breed

rex = Dog("Рекс", "Вівчарка")

rex.name    # "Рекс"
```

self — посилання на той конкретний об'єкт, який ми викликали метод

АТРИБУТИ ЕКЗЕМПЛА

```
class Dog:
    species = "Canis familiaris"    # спільний для всіх

    def __init__(self, name):
        self.name = name            # свій у кожного

rex = Dog("Рекс")
buddy = Dog("Баді")

rex.species == buddy.species    # True
rex.name == buddy.name          # False
```

МЕТОДИ ЕКЗЕМПЛЯ

```
class Dog:

    def __init__(self, name):
        self.name = name

    def bark(self):
        return f"{self.name} каже: Гав!"

rex = Dog("Рекс")
rex.bark()
```

`rex.bark()` — це синтаксичний цу

УСПАДКУВАННЯ ТА S

```
class Animal:

    def __init__(self, name):

        self.name = name

    def speak(self):

        return "..."

class Dog(Animal):

    def __init__(self, name, breed):

        super().__init__(name)

        self.breed = breed

    def speak(self):

        return f"{self.name}: Гав!"
```

ВИКЛИК

```
rex = Dog("Рекс", "Вівчарка")  
rex.speak()    # "Рекс: Гав!"
```

`super()` викликає метод батьків
переписувати вс

ІНКАПСУЛЯЦІЯ ЗА УГ

```
class Account:
    def __init__(self, balance):
        self.balance = balance          # публічний
        self._pin = "1234"              # "protected" — угода
        self.__secret_key = "xyz"       # "private" — name mangling

a = Account(100)
a._pin                                # спрацює
a._Account__secret_key               # теж спрацює — mangling лише ускладнює
```

У C++ `private` перевіряє компілятор
домовленістю

МАГІЧНІ МЕТОДИ: ЩО

```
class Point:

    def __init__(self, x, y):

        self.x, self.y = x, y

    def __repr__(self):

        return f"Point({self.x}, {self.y})"

p = Point(1, 2)

print(p)    # Point(1, 2) — викликає __repr__
```

ПЕРЕВАНТАЖЕННЯ О

```
class Vector:

    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

v1, v2 = Vector(1, 2), Vector(3, 4)
v1 + v2    # викликає v1.__add__(v2)
```

ДЛЯ ПОРІВНЯННЯ —

```
Vector operator+(const Vector& other) const {  
    return Vector{x + other.x, y + other.y};  
}
```

Тут вибір оператора — на етапі компіляції
час виконання

RADD

```
class Vector:

    def __add__(self, other):

        if not isinstance(other, Vector):

            return NotImplemented

        return Vector(self.x + other.x, self.y + other.y)

    def __radd__(self, other):

        return self.__add__(other)
```

Якщо `__add__` повертає `NotImplemented`
то викликається `__radd__` іншого об'єкта

ПОРІВНЯННЯ ОБ'ЄКТ

```
from functools import total_ordering

@total_ordering
class Money:

    def __init__(self, amount):
        self.amount = amount

    def __eq__(self, other):
        return self.amount == other.amount

    def __lt__(self, other):
        return self.amount < other.amount
```

`total_ordering` добудовує $\leq$, $>$,

ПРОТОКОЛ КОЛЕКЦІЙ

```
class Deck:

    def __init__(self):
        self.cards = list(range(52))

    def __len__(self):
        return len(self.cards)

    def __getitem__(self, index):
        return self.cards[index]

    def __contains__(self, card):
        return card in self.cards
```

ВИКОРИСТАННЯ

```
deck = Deck()  
len(deck)          # __len__  
deck[0]            # __getitem__  
for card in deck:  # __getitem__ вмикає ітерацію!  
    ...  
5 in deck          # __contains__
```

CALL — ОБ'ЄКТ ЯК ФУ

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, x):
        return x * self.factor

double = Multiplier(2)
double(21)    # 42
```

PROPERTY — КОНТРОЛЬ ДОСТУПА

```
class Temperature:

    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Ниже абсолютного нуля")
        self._celsius = value
```

ВИГЛЯДАЄ ЯК ПОЛЕ

```
t = Temperature(25)
t.celsius = 30    # насправді викликає setter із валідацією
```

Можна почати з публічного поля, а
— без зміни зовніш

ПОВНИЙ ПРОТОКОЛ

```
class PositiveNumber:

    def __set_name__(self, owner, name):

        self.name = "_" + name

    def __get__(self, instance, owner):

        if instance is None:

            return self

        return getattr(instance, self.name)

    def __set__(self, instance, value):

        if value <= 0:

            raise ValueError(f"{self.name} має бути додатним")

        setattr(instance, self.name, value)
```

ДЕСКРИПТОР ЯК АТР

```
class Product:
    price = PositiveNumber()
    quantity = PositiveNumber()

    def __init__(self, price, quantity):
        self.price = price
        self.quantity = quantity

p = Product(10, 5)
p.price = -5    # ValueError
```

property, staticmethod, classmethod
дескриптор

ФУНКЦІЇ — ОБ'ЄКТ КЛАСУ

```
def greet():  
    print("Привіт!")  
  
def loud(func):  
    def wrapper():  
        print("!!!")  
        func()  
        print("!!!")  
    return wrapper  
  
greet = loud(greet)    # ручне "декорування"
```

СИНТАКСИЧНИЙ ЦУК

```
def loud(func):  
    def wrapper(*args, **kwargs):  
        print("!!!")  
        result = func(*args, **kwargs)  
        print("!!!")  
        return result  
    return wrapper  
  
@loud  
def greet(name):  
    print(f"Привіт, {name}!")
```

@loud над def greet = greet = 1

FUNCTOOLS.WRAPS

```
from functools import wraps

def loud(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@loud
def greet(name): ...

greet.__name__  # 'greet', a He 'wrapper'
```

ДЕКОРАТОР 3 АРГУМЕНТА

```
def repeat(times):  
    def decorator(func):  
        def wrapper(*args, **kwargs):  
            for _ in range(times):  
                func(*args, **kwargs)  
        return wrapper  
    return decorator  
  
@repeat(times=3)  
def hello():  
    print("Привіт!")
```

ДЕКОРАТОР КЛАСУ

```
def singleton(cls):  
    instances = {}  
    def get_instance(*args, **kwargs):  
        if cls not in instances:  
            instances[cls] = cls(*args, **kwargs)  
        return instances[cls]  
    return get_instance  
  
@singleton  
class Config:  
    def __init__(self):  
        self.settings = {}
```

ВБУДОВАНІ ДЕКОРАТ

```
class Circle:

    def __init__(self, radius):
        self.radius = radius

    @staticmethod
    def is_valid_radius(r):
        return r > 0

    @classmethod
    def unit_circle(cls):
        return cls(radius=1)
```

LRU_CACHE

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

ПРОБЛЕМА УПР РЕСУРСА

- Файли, з'єднання, м'ютекси тр
- Refcounting звільняє об'єкт одр
МОВИ
- Циклічні посилання чекають на
непередбачуваний
- `__del__` для ресурсів — погана
явний `with`

ДЛЯ ПОРІВНЯННЯ —

```
class FileHandle {  
    FILE* f;  
public:  
    FileHandle(const char* path) { f = fopen(path, "r"); }  
    ~FileHandle() { if (f) fclose(f); }  
};
```

Деструктор гарантовано звільняє
скоупу — нея

ENTER / EXIT

```
class ManagedFile:

    def __init__(self, path):

        self.path = path

    def __enter__(self):

        self.file = open(self.path, "r")

        return self.file

    def __exit__(self, exc_type, exc_value, traceback):

        self.file.close()

        return False
```

ВИКОРИСТАННЯ

```
with ManagedFile("data.txt") as f:  
    content = f.read()  
# файл гарантовано закритий тут
```

Явний протокол — на відміну від н
+.

ПРИГНІЧЕННЯ ВИНЯТ

```
class IgnoreZeroDivision:

    def __enter__(self):

        return self

    def __exit__(self, exc_type, exc_value, traceback):

        return exc_type is ZeroDivisionError

with IgnoreZeroDivision():

    1 / 0

print("Продовжуємо – виняток придушено")
```

`__exit__` → True = виняток "проко

CONTEXTLIB.CONTEXT

```
from contextlib import contextmanager

@contextmanager
def managed_file(path):
    f = open(path, "r")
    try:
        yield f
    finally:
        f.close()

with managed_file("data.txt") as f:
    content = f.read()
```

До `yield` — `__enter__`, після — `__`

__name__	__doc__
Клас та <code>__init__</code>	Шаблон для посилання на батьківський клас
Атрибути	Екземпляра
Успадкування	<code>super()</code> — дідусь батьківського класу
Інкапсуляція	<code>_protected/underscore</code> та <code>__private/dunder</code> імені, <code>name mangling</code>
Магічні методи	Оператори та <code>dunder</code>
<code>property / descriptor</code>	Синтаксис пошуку атрибутів

Тема	Ключова ідея
Декоратори	Рантайм-обг через @
with	Явний прото